

AI506: Advanced Machine Learning

Project 1: Cat & Dog Classification

Members and Contributions:

Members	Contributions
Nicholas Jason	Data loader pipeline, Data augmentation strategy, Model architecture & Regularization, Results visualization
Stefani Gifita Ganda	Data augmentation strategy, Model architecture & Hyperparameter tuning, Results visualization, Report proofreading
Yiping Zhang	Data augmentation strategy, Model architecture & Optimizer, Report writing, Report proofreading

Table of Contents:

Task 1 - Creating the dataloader	2
Data Loader Structure	2
On-the-fly Data Augmentation	2
Normalization	2
Shuffling of Input Data	2
Task 2 - Constructing the network	2
Model Architecture	2
Model Complexity	3
Global Average Pooling (GAP)	3
Activation Function	3
Regularization (Dropout, BatchNorm)	3
Optimizer (AdamW)	3
Tracking Best Performance and Early Stopping Technique	3
Hyperparameter Tuning using Optuna Study	3
Task 3 - Visualization of results	4
Summary of Training	5
Learning Curves	5
Learned First-Layer Filters	5
Feature Maps per Convolution Block	5
Misclassified Samples	6
Confusion Matrix	6
Bonus Task (Testing Pretrained EfficientNet-B0)	6
Appendix (Code)	7
Task 1 & 2	7
Task 3	12
Bonus task	15

Task 1 - Creating the dataloader

To handle the huge dataset of 3600 jpeg images, we implemented a data loading pipeline using the strategies below.

Data Loader Structure

We utilised `torchvision.datasets.ImageFolder` as suggested in the project hint. This was efficient as the data was already organised into test, train, and validation directories.

On-the-fly Data Augmentation

On-the-fly data augmentation refers to a technique where modified versions of the training data is created during the training process, instead of creating and saving all the augmented data beforehand.

We applied `RandomResizedCrop()`, `RandomHorizontalFlip()`, `RandomRotation()`, and `ColorJitter`. These image transformations will train our model to be translation invariant and scale invariant, ensuring that it can still recognise a cat or dog regardless of its position, orientation, or lighting conditions. By slightly altering the images in each epoch, we prevent the model from memorising specific pixel locations, improving generalization. This also helps to increase the amount of training data to prevent overfitting.

Normalization

The pixel values of the training data were normalized using the ImageNet `MEAN [0.485, 0.456, 0.406]` and `STD [0.229, 0.224, 0.225]`, to ensure that the distribution of input data has zero mean and unit variance. This keeps the initial activations within a range that avoids activation function saturation, hence preventing the possibility of diminishing gradients and ensuring stable weight updates during the initial phases of training.

Shuffling of Input Data

For the input data during training, we set `shuffle=True` to randomise the order of images in each epoch so that the model doesn't generalize over a specific sequence of classification.

Task 2 - Constructing the network

We designed a Convolutional Neural Network (CNN) named CatDogCNN.

Model Architecture

CatDogCNN utilises the Double-Convolution Blocks `DoubleConvBlock` where two 3x3 convolutions are applied before each pooling layer. As discussed in the Convolutional Networks lecture, stacking two 3x3 filters gives a receptive field equivalent to one 5x5 filter but with fewer parameters and more non-linearity. Having two layers instead of one allows for the application of ReLU activation function twice, enabling the network to learn much more complex, non-linear features like the specific shapes and curves of animal body parts. Stacking layers also allows the first convolution to find the edges, and the second to

combine them into textures before downsampling via pooling. `nn.MaxPool2d(2, 2)` reduces the height and width of the images by half, helping the model focus on high-level features.

Model Complexity

We made use of a hierarchical feature extraction pattern. This helps the model break down the complexities by reducing spatial resolution from 128px to 8px. On the other hand, the feature depth expands from 32 to 256. This allows the model to learn low-level features like edges in the early layers, and high-level complex features like distinct animal body parts in the deeper layers that are then condensed via GAP for the final convolutional stage.

Global Average Pooling (GAP)

Instead of flattening the 8 x 8 x 256 feature map into a massive vector, GAP allows us to average each feature map into a single value which greatly reduces the total number of parameters in the classifier, significantly lowering the risk of overfitting.

Activation Function

We used ReLU for all hidden layers and the classifier head. ReLU was highlighted in the Feedforward Networks lecture as a preferred activation function as its non-saturating gradient leads to faster convergence than Sigmoid or Tanh. We believe it provides the necessary non-linearity capabilities for the model to learn complex animal features.

Regularization (Dropout, BatchNorm)

`BatchNorm2d` is added after every convolution to stabilise and accelerate the training process by normalizing the next layer inputs. `Dropout(p=0.5)` layer is used in the fully connected classifier to randomly zero out 50% of the neurons during training, preventing the model from becoming overly reliant on specific neurons. This is a form of anti-overfitting discussed in the Regularization lecture.

Optimizer (AdamW)

We selected the AdamW optimizer with a learning rate of 0.001 and `ReduceLROnPlateau` scheduling. AdamW is able to decouple weight decay from the gradient update for better regularization performance as compared to Adam. The scheduler automatically reduces the learning rate when the model stops improving, a strategy discussed in Hyperparameter Dynamics. This helps the model to fine-tune its weights towards the end of the training.

Tracking Best Performance and Early Stopping Technique

We check whether the model has achieved its best performance after each epoch, by monitoring the validation loss using `improved = vl_loss < best_val_loss`, and saving the weights if applicable. This helps to keep track of the model's best weights after the training process has ended, not just the latest update. If the model does not improve after 5 consecutive epochs, we exercise early stopping, and stop the training early to prevent the model from memorising the training images and causing overfitting.

Hyperparameter Tuning using Optuna Study

At first, we used these default hyperparameters:

Hyperparameter	Value	Justification
IMG_SIZE	128	We thought 128x128 is large enough to understand key features of cats and dogs while allowing reduced memory and fast training.
BATCH_SIZE	32	We avoided too small or too large batch sizes, so we chose somewhere between 16 and 64 that can speed training and save GPU memory without overfitting.
NUM_EPOCHS	30	Initially, we tried other epochs (5 and 20). The accuracy kept increasing, but the training slowed down, so we kept it at 30 for optimal speed as well as accuracy, and used early stopping along the way.
LR	0.001	This is Adam optimizer's default learning rate. We believe small learning rates stabilize the small batch size.
WEIGHT_DECAY	$1e^{-4}$	Since we have over 1M trainable parameters with only 2K training images, we used weight decay to prevent overfitting. The value $1e^{-4}$ is a penalty to the loss function (L^2 norm), keeping the weights small and preventing a single pixel feature from dominating the decision-making.
PATIENCE	5	For stability and efficiency, we wanted the training to stop after 5 epochs to escape from too long of a local plateau.
NUM_WORKERS	2	Two workers match typical CPU core count for data loading. Higher values can cause CPU overhead.

However, we believed there would be better hyperparameters we could implement. Since this is a neural network model, we decided to tweak the learning rate ($1e^{-5}$ to $1e^{-2}$), batch size (16, 32, 64), weight decay ($1e^{-5}$ to $1e^{-3}$), and patience (3 to 7). Those hyperparameters define how the network learns and generalizes. We kept the number of epochs fixed because the early stopping handled effective epochs. After over an hour of running Optuna study, it turned out that the best hyperparameters were as below with final validation loss of 0.4403.

```
batch_size: 32
lr: 0.00011701416881131436
weight_decay: 0.0005674118614493388
patience: 6
```

Task 3 - Visualization of results

In this section, we analyse the internal representations and performance of our CatDogCNN.

Summary of Training

Before hyperparameter tuning:

- Lowest validation loss: 0.5596 (Epoch 29)
- Test accuracy: 73.00%

Weights from Epoch 29 were saved as the best-performing, and used for final test evaluation on the test dataset.

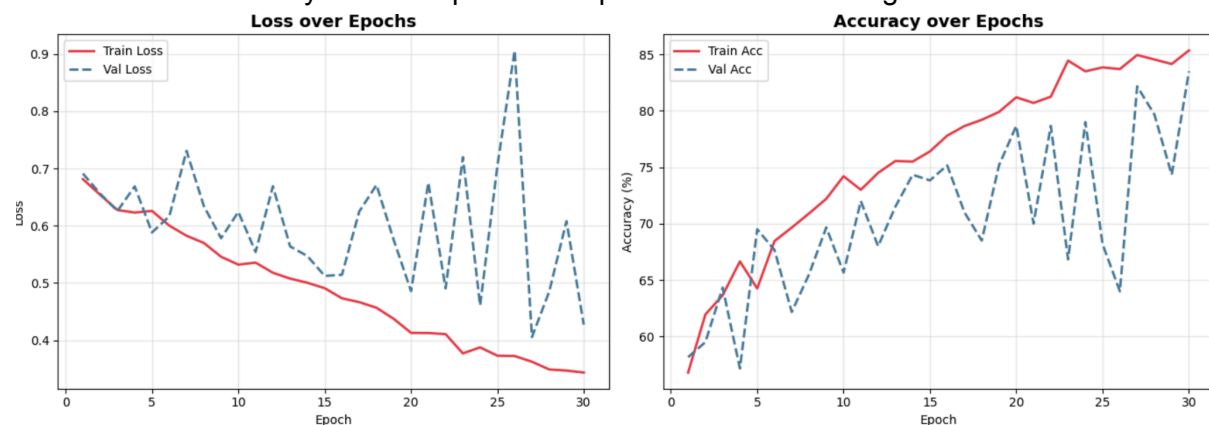
After hyperparameter tuning:

- Lowest validation loss: 0.4056 (Epoch 30)
- Test accuracy: 84.25%

As we can see, CatDogCNN performed better after hyperparameter tuning (accuracy cranked up by 11%, validation loss dropped by ~ 0.1). However, more epochs could've been implemented for higher accuracy, but sadly we didn't have enough time to train further.

Learning Curves

The Loss and Accuracy over 30 Epochs was plotted for both training and validation sets.

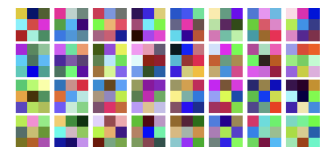


First, we observe that the training loss mostly follows a consistent downward trend, while the validation loss, though generally decreasing, exhibits more frequent spikes and volatility. The divergence between training and validation loss, from epoch 15, suggests that the model is reaching its generalization limit although it is successfully learning.

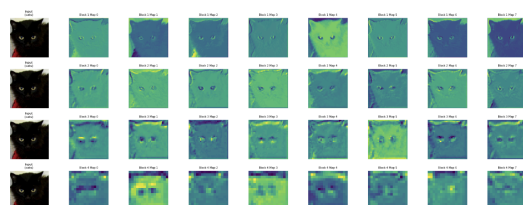
Next, we observe that the validation accuracy follows a similar upward trend with the training accuracy. A generalization gap also emerges around epoch 15. The high oscillations in the validation set graphs are likely due to the difficulty of the task and the small batch size.

Learned First-Layer Filters

The first-layer filters look like edge detectors and colour blobs. Such is the standard hierarchy of a CNN. Low-level features are learned before moving onto more complex ones.



Feature Maps per Convolution Block



We visualised the activations of an input cat image as it passed through the DoubleConvBlock stages.

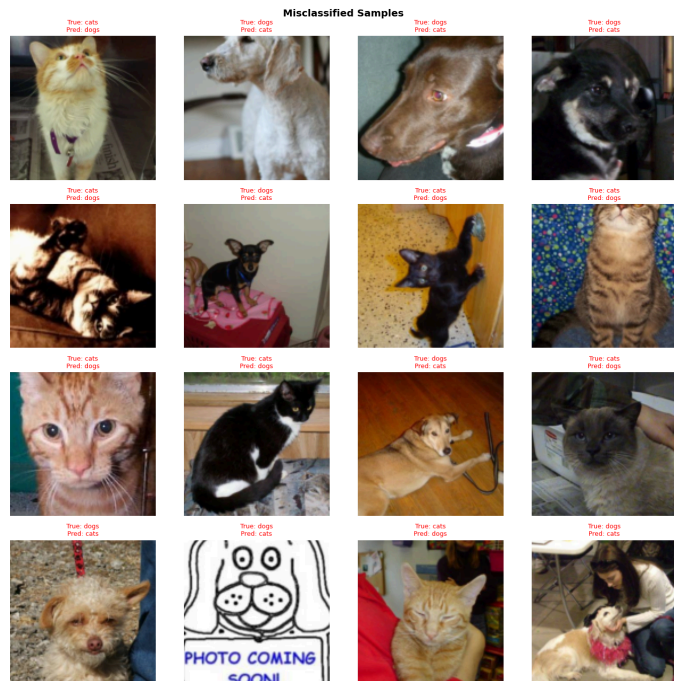
In blocks 1 and 2, the feature maps still clearly resemble a cat image, focusing on the edges and

texture of the image. In blocks 3 and 4, the image becomes more abstract, and represent high-level semantic features. This shows that the deeper layers are discarding redundant spatial information like the background, to focus on the patterns that define a cat.

Misclassified Samples

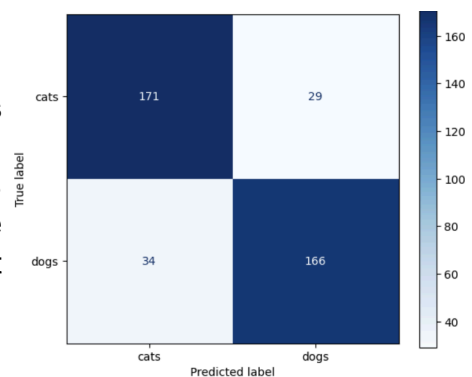
The test loader was shuffled randomly before generating the misclassified samples. This helps to confirm that the shown samples are fairly distributed across both classes.

We observed some patterns in the misclassified images. First, presence of close-up images, where the full animal is not present. Second, images where humans are holding the animals, which can add “noise” to the shape of the animal. Third, images where the animals are in ambiguous poses, such as being curled up or seen from odd angles. Fourth, images of drawn animals, which clearly have different features than real photos. These errors suggest that the model’s weakness is in poses or shapes not heavily represented in the training set.



Confusion Matrix

The confusion matrix shows a balanced performance on both cats and dogs. There are 171 cats and 166 dogs being correctly identified, with 29 cats and 34 dogs being wrongly identified. We can conclude that the model does not have significant bias towards either one of the classes. This is because the dataset used was not imbalanced, and the images were of high quality.



Bonus Task

(Testing Pretrained EfficientNet-B0)

The pretrained model EfficientNet-B0 that we chose randomly could achieve a test accuracy of 92.5%, having a better performance than our custom CNN’s 84.25% (after hyperparameter tuning). This comparison highlights the power of transfer learning. While our custom CNN had to learn basic features from a set of limited training images, the EfficientNet-B0 already had a rich understanding of visual features learned from the massive ImageNet dataset. This allowed the pretrained model to reach a higher accuracy.

Appendix (Code)

Code is available to view in the Colab: [AI506 Advanced ML Code - Project Assignment 1](#)

Task 1 & 2

This is the hyperparameter tuning code which includes most of the functions (copied from the initial training code blocks) for Task 1 and Task 2.

```
import torch.nn as nn
import torch.optim as optim
import copy
import os
import optuna

import torch
import random
import numpy as np
import torchvision.transforms as transforms
from torchvision.datasets import ImageFolder
from torch.utils.data import DataLoader

# below code are copied from earlier cells

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
torch.backends.cudnn.deterministic = True

if torch.cuda.is_available():
    DEVICE = torch.device("cuda")
elif torch.backends.mps.is_available():
    DEVICE = torch.device("mps")
else:
    DEVICE = torch.device("cpu")

SAVE_DIR = "/content/drive/MyDrive/Colab Notebooks/Advanced ML SDU - Project 1 (1)"
DATA_DIR = os.path.join(SAVE_DIR, "catdog_data")
IMG_SIZE = 128

# Default Hyperparameters
DEFAULT_HYPERPARAMS = {
    "BATCH_SIZE": 32,
    "NUM_EPOCHS": 30,
```

```

    "LR": 1e-3,
    "WEIGHT_DECAY": 1e-4,
    "PATIENCE": 5,
}

```

```

NUM_WORKERS = 2

```

```

MEAN = [0.485, 0.456, 0.406]

```

```

STD = [0.229, 0.224, 0.225]

```

```

# Data Loaders (Task 1)

```

```

train_transforms = transforms.Compose([
    transforms.RandomResizedCrop(IMG_SIZE, scale=(0.8, 1.0)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(degrees=15),
    transforms.ColorJitter(brightness=0.3, contrast=0.3, saturation=0.2, hue=0.05),
    transforms.ToTensor(),
    transforms.Normalize(mean=MEAN, std=STD),
])

```

```

val_test_transforms = transforms.Compose([
    transforms.Resize(IMG_SIZE + 16),
    transforms.CenterCrop(IMG_SIZE),
    transforms.ToTensor(),
    transforms.Normalize(mean=MEAN, std=STD),
])

```

```

def get_dataloaders(data_dir, batch_size=DEFAULT_HYPERPARAMS["BATCH_SIZE"]):
    train_set = ImageFolder(os.path.join(data_dir, "train"), transform=train_transforms)
    val_set = ImageFolder(os.path.join(data_dir, "validation"), transform=val_test_transforms)
    test_set = ImageFolder(os.path.join(data_dir, "test"), transform=val_test_transforms)

    train_loader = DataLoader(train_set, batch_size=batch_size, shuffle=True,
                              num_workers=NUM_WORKERS, pin_memory=True)
    val_loader = DataLoader(val_set, batch_size=batch_size, shuffle=False,
                            num_workers=NUM_WORKERS, pin_memory=True)
    test_loader = DataLoader(test_set, batch_size=batch_size, shuffle=False,
                             num_workers=NUM_WORKERS, pin_memory=True)

    return train_set, val_set, test_set, train_loader, val_loader, test_loader

```


Model (Task 2)

```
class DoubleConvBlock(nn.Module):
    """Conv → BN → ReLU → Conv → BN → ReLU → MaxPool"""
    def __init__(self, in_ch, out_ch):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, kernel_size=3, padding=1),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_ch, out_ch, kernel_size=3, padding=1),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2, 2),
        )

    def forward(self, x):
        return self.block(x)

class CatDogCNN(nn.Module):
    def __init__(self, num_classes=2):
        super().__init__()

        self.features = nn.Sequential(
            DoubleConvBlock(3, 32), # 128 × 128 → 64 × 64
            DoubleConvBlock(32, 64), # 64 × 64 → 32 × 32
            DoubleConvBlock(64, 128), # 32 × 32 → 16 × 16
            DoubleConvBlock(128, 256), # 16 × 16 → 8 × 8
        )

        # Global Average Pooling: collapses (256, 8, 8) → (256,)
        self.gap = nn.AdaptiveAvgPool2d((1, 1))

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(256, 128),
            nn.ReLU(inplace=True),
            nn.Dropout(p=0.5),
            nn.Linear(128, num_classes),
        )

    def forward(self, x):
```

```

x = self.features(x)
x = self.gap(x)
x = self.classifier(x)
return x

```

```

def count_params(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

```

```

def train_one_epoch(model, loader, criterion, optimizer):
    model.train()
    running_loss, correct, total = 0.0, 0, 0
    for imgs, labels in loader:
        imgs, labels = imgs.to(DEVICE), labels.to(DEVICE)
        optimizer.zero_grad()
        outputs = model(imgs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item() * imgs.size(0)
        _, preds = outputs.max(1)
        correct += preds.eq(labels).sum().item()
        total += labels.size(0)
    return running_loss / total, correct / total

```

```

@torch.no_grad()
def evaluate(model, loader, criterion):
    model.eval()
    running_loss, correct, total = 0.0, 0, 0
    all_preds, all_labels = [], []
    for imgs, labels in loader:
        imgs, labels = imgs.to(DEVICE), labels.to(DEVICE)
        outputs = model(imgs)
        loss = criterion(outputs, labels)
        running_loss += loss.item() * imgs.size(0)
        _, preds = outputs.max(1)
        correct += preds.eq(labels).sum().item()
        total += labels.size(0)
        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())
    return running_loss / total, correct / total, all_preds, all_labels

```

```

def train_model_for_optuna(model, train_loader, val_loader, trial, params):

```

```

criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(model.parameters(), lr=params["LR"],
weight_decay=params["WEIGHT_DECAY"])
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode="min",
factor=0.5, patience=params["PATIENCE"])

```

```

history = {"train_loss": [], "train_acc": [], "val_loss": [], "val_acc": []}
best_val_loss = float("inf")

```

```

for epoch in range(1, params["NUM_EPOCHS"] + 1):
    tr_loss, tr_acc = train_one_epoch(model, train_loader, criterion, optimizer)
    vl_loss, vl_acc, _ = evaluate(model, val_loader, criterion)
    scheduler.step(vl_loss)

```

```

history["train_loss"].append(tr_loss)
history["train_acc"].append(tr_acc)
history["val_loss"].append(vl_loss)
history["val_acc"].append(vl_acc)

```

```

if vl_loss < best_val_loss:
    best_val_loss = vl_loss

```

```

trial.report(vl_loss, epoch)

```

```

if trial.should_prune():
    raise optuna.exceptions.TrialPruned()

```

```

print(
    f"Epoch [{epoch:02d}/{params['NUM_EPOCHS']}] "
    f"Train Loss: {tr_loss:.4f} | Train Acc: {tr_acc*100:.2f}% | "
    f"Val Loss: {vl_loss:.4f} | Val Acc: {vl_acc*100:.2f}%"
)

```

```

return best_val_loss

```

Optuna Objective Function

```

def objective(trial):
    hparams = {
        "BATCH_SIZE": trial.suggest_categorical("batch_size", [16, 32, 64]),
        "NUM_EPOCHS": DEFAULT_HYPERPARAMS["NUM_EPOCHS"], # Fixed, as early stopping
handles effective epochs

```

```

    "LR": trial.suggest_loguniform("lr", 1e-5, 1e-2),
    "WEIGHT_DECAY": trial.suggest_loguniform("weight_decay", 1e-5, 1e-3),
    "PATIENCE": trial.suggest_int("patience", 3, 7),
}

_, _, _, train_loader, val_loader, _ = get_dataloaders(DATA_DIR,
batch_size=hparams["BATCH_SIZE"])

model = CatDogCNN(num_classes=2).to(DEVICE)

val_loss = train_model_for_optuna(model, train_loader, val_loader, trial, hparams)

return val_loss

```

Task 3

Visualisations (Task 3)

```

def plot_training_curves(history):
    """3a — Loss and accuracy curves."""
    epochs = range(1, len(history["train_loss"]) + 1)
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

    ax1.plot(epochs, history["train_loss"], label="Train Loss", color="#E63946", linewidth=2)
    ax1.plot(epochs, history["val_loss"], label="Val Loss", color="#457B9D", linewidth=2,
linestyle="--")
    ax1.set_title("Loss over Epochs", fontsize=14, fontweight="bold")
    ax1.set_xlabel("Epoch"); ax1.set_ylabel("Loss")
    ax1.legend(); ax1.grid(alpha=0.3)

    ax2.plot(epochs, [a * 100 for a in history["train_acc"]], label="Train Acc", color="#E63946",
linewidth=2)
    ax2.plot(epochs, [a * 100 for a in history["val_acc"]], label="Val Acc", color="#457B9D",
linewidth=2, linestyle="--")
    ax2.set_title("Accuracy over Epochs", fontsize=14, fontweight="bold")
    ax2.set_xlabel("Epoch"); ax2.set_ylabel("Accuracy (%)")
    ax2.legend(); ax2.grid(alpha=0.3)

    plt.tight_layout()
    path = os.path.join(VIZ_DIR, "training_curves.png")
    plt.savefig(path, dpi=150, bbox_inches="tight")
    plt.show()
    print(f"Saved: {path}")

```

```

def visualize_filters(model):
    """3b — Learned first-layer filters."""
    # Access first conv inside first DoubleConvBlock
    filters = model.features[0].block[0].weight.data.clone().cpu()
    n = min(32, filters.shape[0])
    fig, axes = plt.subplots(4, 8, figsize=(16, 8))
    for i, ax in enumerate(axes.flat):
        if i < n:
            f = filters[i]
            f = (f - f.min()) / (f.max() - f.min() + 1e-8)
            ax.imshow(f.permute(1, 2, 0).numpy())
        ax.axis("off")
    fig.suptitle("Learned First-Layer Filters", fontsize=16, fontweight="bold", y=1.01)
    plt.tight_layout()
    path = os.path.join(VIZ_DIR, "first_layer_filters.png")
    plt.savefig(path, dpi=150, bbox_inches="tight")
    plt.show()
    print(f"Saved: {path}")

```

```

def visualize_feature_maps(model, dataset, n_maps=8):
    """3c — Feature maps from each conv block."""
    img_tensor, label = dataset[0]
    class_name = dataset.classes[label]
    activations = []
    hooks = []

    # Hook into the first conv of every DoubleConvBlock
    for block in model.features:
        hooks.append(block.block[0].register_forward_hook(
            lambda m, i, o: activations.append(o.detach().cpu())
        ))

    with torch.no_grad():
        model(img_tensor.unsqueeze(0).to(DEVICE))
    for h in hooks:
        h.remove()

    fig, axes = plt.subplots(len(activations), n_maps + 1, figsize=(20, 8))
    denorm = img_tensor.clone()
    for c, (m, s) in enumerate(zip(MEAN, STD)):
        denorm[c] = denorm[c] * s + m

```

```
denorm = denorm.clamp(0, 1).permute(1, 2, 0).numpy()
```

```
for row in range(len(activations)):
    axes[row, 0].imshow(denorm)
    axes[row, 0].set_title(f"Input\n({class_name})", fontsize=8)
    axes[row, 0].axis("off")
    for col in range(n_maps):
        fmap = activations[row][0, col].numpy()
        axes[row, col + 1].imshow(fmap, cmap="viridis")
        axes[row, col + 1].set_title(f"Block {row+1} Map {col}", fontsize=7)
        axes[row, col + 1].axis("off")

fig.suptitle("Feature Maps per Conv Block", fontsize=14, fontweight="bold")
plt.tight_layout()
path = os.path.join(VIZ_DIR, "feature_maps.png")
plt.savefig(path, dpi=150, bbox_inches="tight")
plt.show()
print(f"Saved: {path}")
```

```
def show_misclassified(model, loader, dataset, n=16):
    """3d — Misclassified samples."""
    model.eval()
    mis_imgs, mis_labels, mis_preds = [], [], []
    with torch.no_grad():
        for imgs, labels in loader:
            imgs, labels = imgs.to(DEVICE), labels.to(DEVICE)
            _, preds = model(imgs).max(1)
            mask = preds != labels
            mis_imgs.append(imgs[mask].cpu())
            mis_labels.append(labels[mask].cpu())
            mis_preds.append(preds[mask].cpu())
            if sum(len(m) for m in mis_imgs) >= n:
                break

    mis_imgs = torch.cat(mis_imgs)[:n]
    mis_labels = torch.cat(mis_labels)[:n]
    mis_preds = torch.cat(mis_preds)[:n]
    classes = dataset.classes

    cols = 4
    rows = (n + cols - 1) // cols
```

```

fig, axes = plt.subplots(rows, cols, figsize=(cols * 3.5, rows * 3.5))
for i, ax in enumerate(axes.flat):
    if i < len(mis_imgs):
        img = mis_imgs[i].clone()
        for c, (m, s) in enumerate(zip(MEAN, STD)):
            img[c] = img[c] * s + m
        ax.imshow(img.clamp(0, 1).permute(1, 2, 0).numpy())
        ax.set_title(
            f"True: {classes[mis_labels[i]]}\nPred: {classes[mis_preds[i]]}",
            color="red", fontsize=9
        )
    ax.axis("off")
fig.suptitle("Misclassified Samples", fontsize=14, fontweight="bold")
plt.tight_layout()
path = os.path.join(VIZ_DIR, "misclassified.png")
plt.savefig(path, dpi=150, bbox_inches="tight")
plt.show()
print(f"Saved: {path}")

```

```

def plot_confusion_matrix(preds, labels, class_names):
    """3e — Confusion matrix (from Model B)."""
    from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
    cm = confusion_matrix(labels, preds)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=class_names)
    fig, ax = plt.subplots(figsize=(6, 5))
    disp.plot(cmap="Blues", ax=ax)
    ax.set_title("Confusion Matrix", fontsize=14, fontweight="bold")
    plt.tight_layout()
    path = os.path.join(VIZ_DIR, "confusion_matrix.png")
    plt.savefig(path, dpi=150, bbox_inches="tight")
    plt.show()
    print(f"Saved: {path}")

```

Bonus task

BONUS — Fine-tuning EfficientNet-B0 (timm)

```

def run_efficientnet_bonus(train_loader, val_loader, test_loader, criterion):
    try:
        import timm
    except ImportError:

```

```

    print("\ntimm not installed — run: pip install timm")
    return None

FROZEN_EPOCHS = 5
UNFROZEN_EPOCHS = 15
LR_HEAD = 1e-3
LR_FULL = 1e-4

pt_model = timm.create_model("efficientnet_b0", pretrained=True, num_classes=2).to(DEVICE)
print(f"\n=== BONUS: EfficientNet-B0 ( {count_params(pt_model):,} params) ===")

# Stage 1 (train only the new classifier head at a high LR)
for name, param in pt_model.named_parameters():
    if "classifier" not in name:
        param.requires_grad = False
opt_pt = optim.Adam(filter(lambda p: p.requires_grad, pt_model.parameters()), lr=LR_HEAD)

print("[Stage 1] Training head only...")
for epoch in range(1, FROZEN_EPOCHS + 1):
    train_one_epoch(pt_model, train_loader, criterion, opt_pt)
    _, v_acc, _, _ = evaluate(pt_model, val_loader, criterion)
    print(f" Epoch [{epoch}/{FROZEN_EPOCHS}] Val Acc: {v_acc*100:.2f}%")

# Stage 2 (unfreeze everything and fine-tune at a low LR (1e-4) with CosineAnnealingLR for smooth
LR decay)
for param in pt_model.parameters():
    param.requires_grad = True
opt_pt = optim.Adam(pt_model.parameters(), lr=LR_FULL, weight_decay=1e-4)
sch_pt = optim.lr_scheduler.CosineAnnealingLR(opt_pt, T_max=UNFROZEN_EPOCHS)

print("[Stage 2] Fine-tuning all layers...")
best_pt_acc = 0.0
pt_path = os.path.join(SAVE_DIR, "best_efficientnet.pth")
for epoch in range(1, UNFROZEN_EPOCHS + 1):
    train_one_epoch(pt_model, train_loader, criterion, opt_pt)
    _, v_acc, _, _ = evaluate(pt_model, val_loader, criterion)
    sch_pt.step()
    if v_acc > best_pt_acc:
        best_pt_acc = v_acc
        torch.save(pt_model.state_dict(), pt_path)
    print(f" Epoch [{epoch}/{UNFROZEN_EPOCHS}] Val Acc: {v_acc*100:.2f}%")

```



```
pt_model.load_state_dict(torch.load(pt_path, map_location=DEVICE))
_, pt_test_acc, _, _ = evaluate(pt_model, test_loader, criterion)
print(f"\nEfficientNet-B0 Test Accuracy: {pt_test_acc*100:.2f}%")
return pt_test_acc
```